



Shree Rayeshwar Institute of Engineering & IT

Department of Computer Engineering

Date: 24/08/2026

PROJECT SYNOPSIS

1. Project Title/Idea: Autonomous Remediation of Declarative Cloud Configurations.

2. Team Member Details:

Sr. No	Roll no	Name	Email_Id	Mobile Number	Signature
1	C21 - 52	Yash Naik Gaonkar	gaonkaryash4306@gmail.com	7219006865	
2	C23 - 36	Ramzan Ansari	ansariramzan2004@gmail.com	9823211848	
3	C22 - 50	Shailesh Sawant	sawantshailesh560@gmail.com	8767982647	
4	C23 - 31	Nikki Sharma	sharmaniks2004@gmail.com	7722083537	

3. Introduction:

Modern cloud environments rely on Infrastructure as Code (IaC) to manage resources. Primarily utilizing HashiCorp Configuration Language (HCL) via Terraform or Pulumi. However, a critical failure point exists: when developers manually alter live cloud configurations during emergencies, the live state "drifts" away from the source code. Current industry tools only flag these discrepancies, leaving Cloud Engineering and DevOps teams to manually reverse-engineer the

changes and write corrective code, a process taking hours. This project proposes building an autonomous AI Agent that solves this exact problem. The system will continuously monitor cloud states, detect configuration drift, optimize resources, and automatically generate syntactically valid IaC code to repair the source repository. The scope extends beyond simple error detection to full closed-loop remediation, covering drift correction, proactive cost optimization, security vulnerability patching, and CI/CD pipeline infrastructure validation.

4. Literature survey:

An extensive review of existing cloud automation and Site Reliability Engineering (SRE) methodologies was conducted to select this approach.

Existing Methodologies: Standard drift detection tools like driftctl and terraform plan were analyzed. While highly effective at identifying JSON state discrepancies, they inherently lack remediation capabilities, requiring manual human intervention.

AI Integration: Research into generic Large Language Models (LLMs) demonstrated that while models can generate boilerplate code, they fail in DevOps without real-time infrastructure context.

Proposed Advancement: By reviewing documentation on Parameter-Efficient Fine-Tuning (PEFT) and advanced Prompt Engineering, this project addresses the existing gap. Instead of relying on passive monitoring, the methodology uses API-driven contextual pipelines to feed live state data into an agent, constraining it to generate valid, state-aware infrastructure code.

5. Problem Statement and Objective:

Problem Statement

Cloud infrastructure suffers from undocumented manual modifications and inefficiencies, leading to configuration drift, exposed security vulnerabilities, and over-provisioned resource costs. Existing IaC tools detect these issues but require manual, time-consuming coding efforts to bring the source code back into synchronization with reality.

Objectives:

1. To develop a Python-based orchestrator that continuously fetches live cloud states using standard SDKs.
2. To detect and diagnose discrepancies between the desired IaC blueprint and live cloud infrastructure.
3. To utilize an AI reasoning engine to autonomously generate syntactically correct IaC code fixes.

-
4. To establish a secure, human-in-the-loop Git automation workflow that submits auditable Pull Requests for peer review instead of applying infrastructure updates directly.

6. Project Features and Outcome:

Features:

1. **Autonomous Drift Remediation:** Automatically detects live cloud changes and writes the corresponding Terraform code to update the repository.
2. **Automated Security Remediation:** Scans for misconfigurations (e.g., exposed ports) and autonomously drafts PRs to enforce zero-trust security limits.
3. **Proactive Cost Optimization:** Analyzes resource utilization metrics and suggests code changes to downgrade underutilized servers, calculating monthly savings.
4. **Intelligent Validation:** Intercepts CI/CD pipeline deployments to automatically append missing organizational tags before provisioning.

Outcome:

At completion, the project will demonstrate a fully functional AI Agent operating in a simulated cloud environment. It will showcase a closed-loop automated workflow where manual cloud changes are detected, repaired in code, and safely presented to an engineer for final approval via a Pull Request within seconds.

7. Methodology & Proposed Process Development:

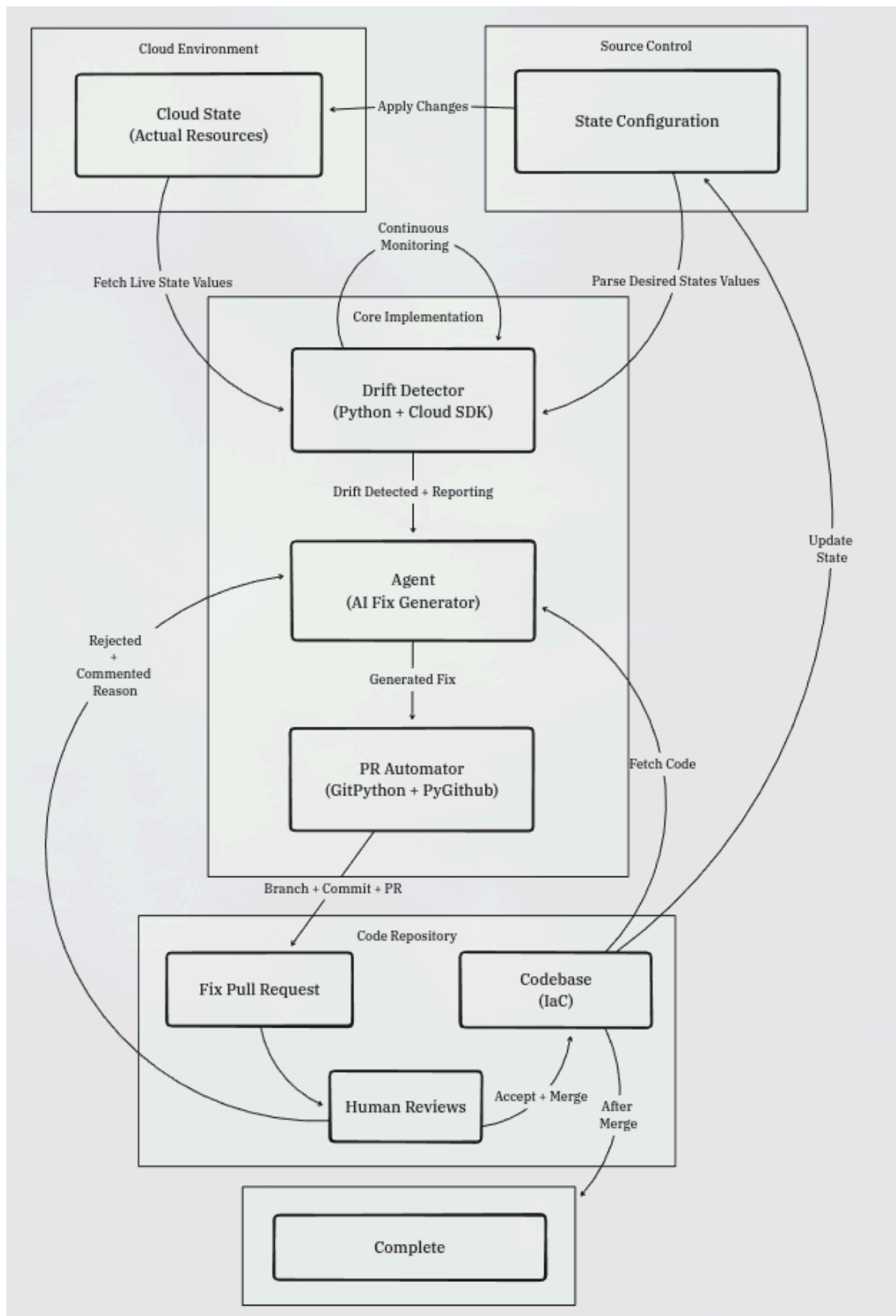
Methodology:

The proposed system is structured around a three-tier architectural framework comprising "Observation, Analysis & Execution."

1. **Observation:** A Python orchestration layer utilizes Cloud SDKs to programmatically fetch live infrastructure data and compare it against the JSON representations of terraform.tfstate.
2. **Analysis:** Discrepancies are formatted into structured Drift Reports. Instead of training a massive foundational model from scratch, the system utilizes highly structured Prompt Engineering (or LoRA fine-tuning) to feed this live context into an agent. The AI is constrained to act solely as a translation layer, reverse-engineering the JSON diff back into valid Terraform HCL blocks.
3. **Execution:** Utilizing GitPython and PyGithub, the agent autonomously creates a version-control branch, commits the AI-generated IaC code fixes, and opens a Pull Request to ensure safety and auditability.

Software to be used: Python, IaC(Terraform, Pulumi), Cloud SDKs, Git/GitHub APIs, Docker.

Proposed Execution:



8. References:

- [1] L. Yogesh, N. N, S. Rai, S. Reddy B, and D. V. S., “Infrastructure-as-Code (IaC) Drift Security with GitOps-Driven Auto-Remediation in AWS,” *Journal of Emerging Technologies and Innovative Research (JETIR)*, vol. 12, no. 11, 2025, DOI: 10.56975/jetir.v12i11.571291.
- [2] Z. Yang, H. Guan, V. Nicolet, B. Paulsen, J. Dodds, D. Kroening, and A. Chen, “Automated Cloud Infrastructure-as-Code Reconciliation with AI Agents,” *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2026.
- [3] S. Bhagvanthakur, “Demo: tfdrift – A Severity Taxonomy and Risk Classification Framework for Infrastructure Drift Detection,” *arXiv preprint, arXiv:2608.18173*, 2026.
- [4] A. A. Akintade, “From Detection to Verified Action: Operational Readiness for AI-Enabled Cloud Failure Management,” *International Journal for Research in Applied Science and Engineering Technology (IJRASET)*, 2026, DOI: 10.22214/ijraset.2026.84530.
- [5] N. Saavedra, J. F. Ferreira, and A. Mendes, “InfraFix: Technology-Agnostic Repair of Infrastructure as Code,” *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 41–45, 2025, DOI: 10.1145/3713081.3731735.
- [6] P. T. J. Kon et al., “IaC-Eval: A Code Generation Benchmark for Cloud Infrastructure-as-Code Programs,” *NeurIPS Datasets and Benchmarks*, 2024.
- [7] P. Jana et al., “TerraFormer: Automated Infrastructure-as-Code with LLMs Fine-Tuned via Policy-Guided Verifier Feedback,” *arXiv preprint, arXiv:2601.08734*, 2026.
- [8] R. Nekrasov et al., “IaC Generation with LLMs: An Error Taxonomy and A Study on Configuration Knowledge Injection,” *arXiv preprint, arXiv:2512.14792*, 2025.
- [9] A. Verdet, M. Hamdaqa, L. Da Silva, and F. Khomh, “Exploring Security Practices in Infrastructure as Code: An Empirical Study,” *arXiv preprint, arXiv:2308.03952*, 2023.
- [10] N. Bessghaier et al., “On the Prevalence, Co-occurrence, and Impact of Infrastructure-as-Code Smells,” *IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 23–34, 2024, DOI: 10.1109/SANER60148.2024.00009.

-
- [11] A. Rahman, E. Farhana, C. Parnin, and L. Williams, “Gang of Eight: A Defect Taxonomy for Infrastructure as Code Scripts,” ACM/IEEE International Conference on Software Engineering (ICSE), 2020.
- [12] A. Rahman and L. Williams, “Source Code Properties of Defective Infrastructure as Code Scripts,” Information and Software Technology, 2019, DOI: 10.1016/j.infsof.2019.04.013.
- [13] A.-D. Tran, L. Sion, K. Yskout, and W. Joosen, “TerrARA: Automated Security Threat Modeling for Infrastructure as Code,” ACM Conference on Data and Application Security and Privacy (CODASPY), pp. 269–280, 2025, DOI: 10.1145/3714393.3726494.
- [14] T. Ahmed et al., “Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models,” IEEE/ACM International Conference on Software Engineering (ICSE), pp. 1737–1749, 2023.
- [15] X. Zhang et al., “Automated Root Causing of Cloud Incidents using In-Context Learning with GPT-4,” ACM International Conference on the Foundations of Software Engineering, pp. 266–277, 2024.
- [16] Q. Cheng et al., “AI for IT Operations (AIOps) on Cloud Platforms: Reviews, Opportunities and Challenges,” arXiv preprint, arXiv:2304.04661, 2023.
- [17] S. Ghosh, M. Shetty, C. Bansal, and S. Nath, “How to Fight Production Incidents? An Empirical Study on a Large-scale Cloud Service,” ACM Symposium on Cloud Computing (SoCC), pp. 126–141, 2022, DOI: 10.1145/3542929.3563482.
- [18] M. Begoug, A. Ouni, and M. Chouchen, “How Do Infrastructure-as-Code Practitioners Update Their Dependencies? An Empirical Study on Terraform Module Updates,” IEEE/ACM International Conference on Mining Software Repositories (MSR), pp. 642–653, 2025, DOI: 10.1109/MSR66628.2025.00100.
- [19] F. L. S. Vargas, R. B. Mansilha, and D. Kreutz, “Security-First Evaluation of Text-to-Terraform: Benchmarking LLMs and SLMs for Secure IaC Generation,” arXiv preprint, arXiv:2608.02672, 2026.
- [20] R. N. H. Khan et al., “Multi-Agent Code-Orchestrated Generation for Reliable Infrastructure-as-Code,” arXiv preprint, arXiv:2510.03902, 2025.

